DB12

Normalization

> **Relation shcema를 더 작게 나누어서 더 바람직한 형태가 되도록 하는 작업**

`Normal form` **: Relation의 바람직한 형태에 대한 정의 / 기준**

Normalization의 결과가 가져야 하는 성질

- **Lossless join property: Join을 통해 원래 Table로의 복구가 가능해야 한다.**
  - Spurious tuple의 solution
- **Dependency preservation property: Functional Dependency가 그대로 유지되어야 한다.**
- **쪼갠 결과, 각 Table이 Normal form이어야 한다.**

`Prime attribute` **:** 하나 이상의 Candidate key에 속하는 각 attribute의 집합

First Normal Form (1NF)

> **Table의 각 Attribute가 Atomic value만을 허용하는 경우**

- Single value만 허용되고 Multple value는 허용되지 않는다.
- Relation의 **Formal definition의 일부분**이다.

- Relational model이라면 이미 만족하고 있어야 한다.
- ER to Relational mapping에서도 Atomic하지 않는 Value를 허용하지 않는다.

Non-atomic attributes를 갖도록 하는 **Composite / Multi-valued attribute와 Nested relation은 허용되지 않는다.**

- Nested relation : attribute 값으로 Table 자체를 허용하는 것

Example

(b)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Second Normal Form (2NF)

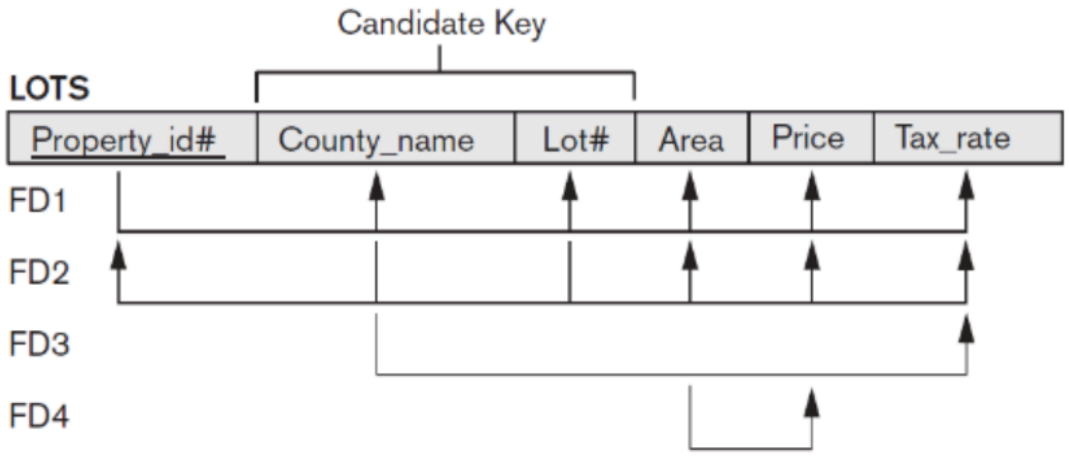
> **1NF를 만족하고, Non-prime attribute인 모든 Attribute A가 모든 Candidate key에 대해 Fully functional dependent한 경우**

- **Partially functional dependent**한 경우에는 R **is not in 2NF**라고 한다.

Full functional dependency : $X \rightarrow Y$**라는 FD에서** X**에 속하는 어떤 Attribute** A**를 제거했을 때, FD가 유지되지 않는 경우**

Partial functional dependency : $X \rightarrow Y$**라는 FD에서** X**에 속하는 어떤 Attribute** A**를 제거했을 때, FD가 유지되는 경우**

Example



위 그림에서 Candidate key는 {Property_id#}, {County_name, Lot#}이다.

- 이로 인해 FD1, FD2가 정의된다.

FD3를 보면, Candidate key에서 Lot#를 제거해도 FD가 유지된다.

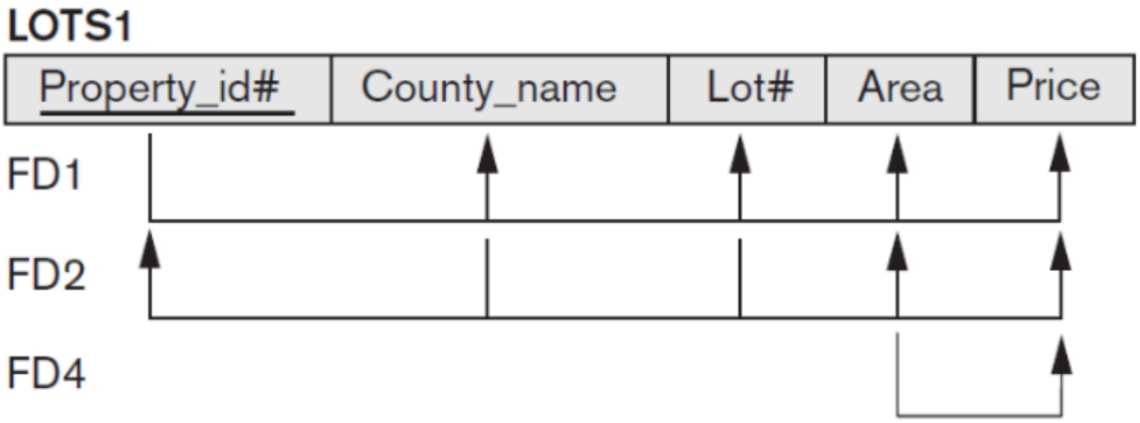
- **Partial functional dependency**

따라서 **LOTS Relation is not in 2NF**이다.

위 상황에서 **2NF가 되도록 Normalization**해보자

- 문제가 되는 **FD3를 테이블 두 개로 나누어 해결**할 수 있다.

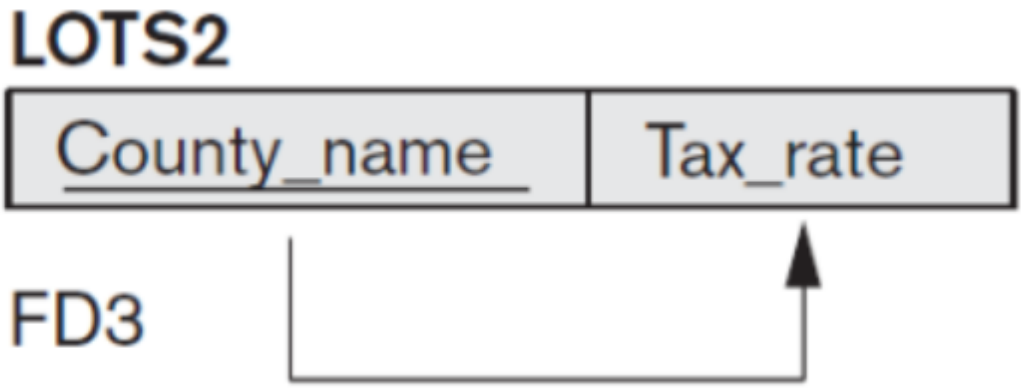
먼저 **LOTS Relation에서** `Tax_rate` **를 제거하여 LOTS1 Relation을 생성**한다.



- FD3가 제거된다.
- $X \rightarrow Y$에서 $R - \{Y - X\}$를 한다고 생각하면 된다.

FD3에서 Lot#를 제거해도 FD가 유지되었기 때문에 **County_name → Tax_rate FD가 유지되도록 하는 Relation LOTS2를 생성**한다.

- FD3의 Attribute가 포함되는 Relation
  - $X \rightarrow Y$에서 $X \cup Y$를 한다고 생각하면 된다.
- **FD에서 X side가 PK가 되도록 한다.**



어떤 1NF를 만족하는 Relation이든 Nomalization process을 통해 반드시 2NF를 만족하도록 할 수 있다.

- 1NF를 만족하지 않는다면 Attibute 자체가 집합 또는 Relation이 되어 FD 정의가 어렵거나 애매해진다.

Third Normal Form (3NF)

> **이전 단계의 NF를 이미 만족할 때, R에서 모든 FD $X \rightarrow A$에 대해 X가 R의 Superkey이거나 A가 R의 Prime attribute인 경우**

- **X가 R의 Superkey:** X가 다 다르면 중복되는 Tuple이 적어진다.
  - **Superkey**: Candidate key보다 최소성이 없는 넓은 개념
- **A가 R의 Prime attribute:** 중복은 발생하나 예외 사항
  - 이 경우에 Normalization을 하면 Relation을 나누게 되어 Candidate key가 깨지기 때문이다.

FD는 중복을 의미한다.

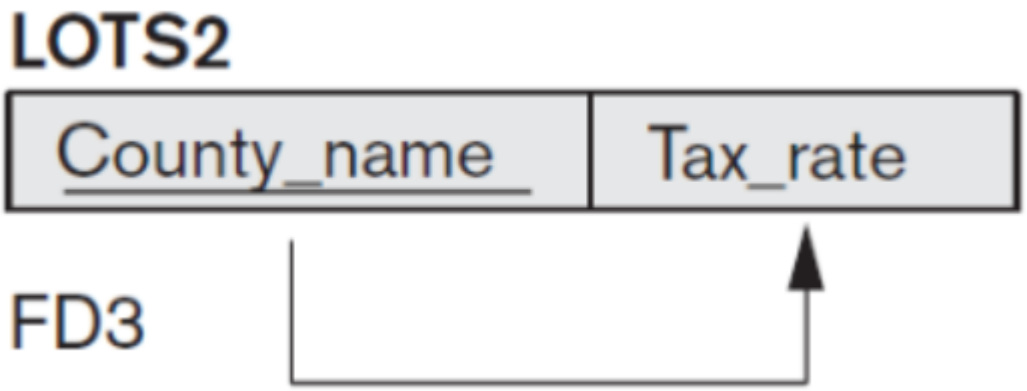
- X와 A가 같은 것이 여러 개 존재한다는 것이다.
- $t_1[X] = t_2[X] \rightarrow t_1[Y] = t_2[Y]$

앞에서 `DB11`에서 배운 것처럼 **Redundunt한 것을 줄이는 것이 중요**하다.

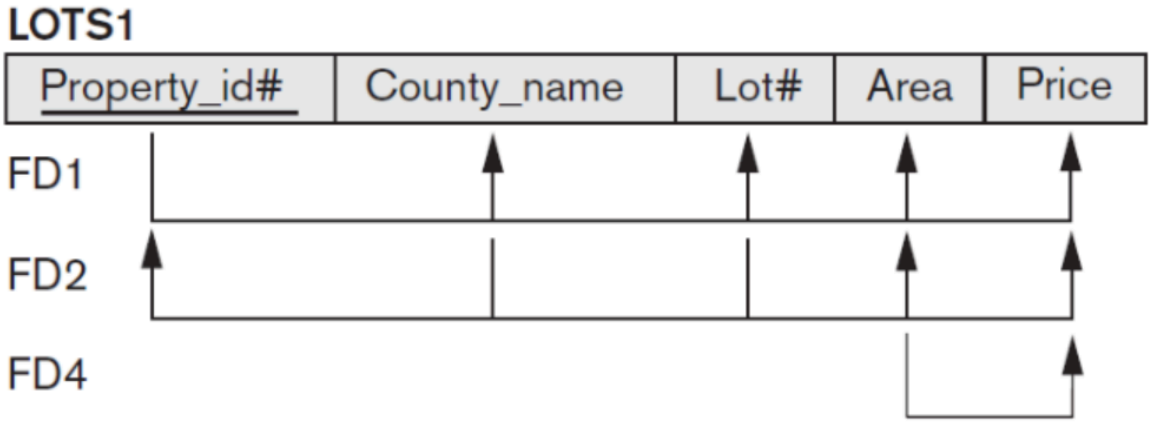
- Update anomaly을 줄이기 위함이다.
- Normalization to 3NF가 필요한 이유
  - X가 Superkey가 되도록 강제하여 중복을 최대한 제거한다.

Example

먼저 **LOTS2** Relation은 FD3에서 X도 Superkey이다. 따라서 **LOTS2 is in 3NF**이다.



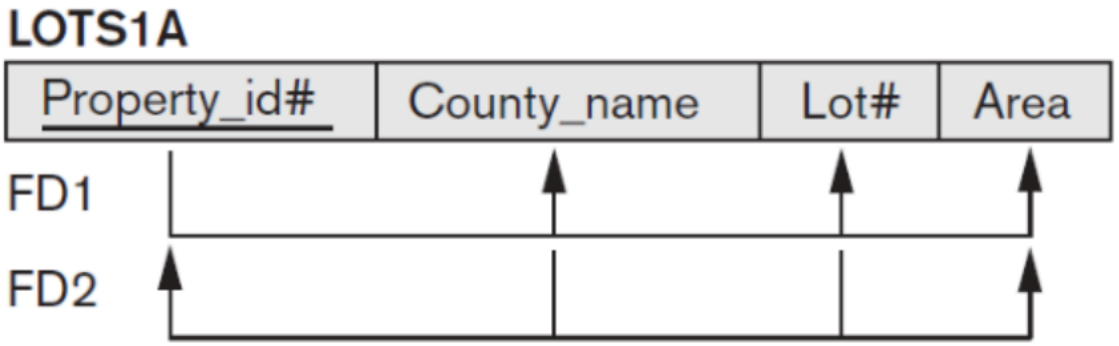
그러나 **LOTS1** Relation은 FD4에서 X도 Superkey가 아니고 A도 Prime attribute가 아니다. 따라서 **LOTS1 is not in 3NF**이다.



LOTS1 Relation을 **Normalizaiton**해보자.

- **LOTS1 Relation을 두 Relation으로 Decompose**한다.

LOTS1A는 FD4를 제거한다.



LOTS1B에서 **FD4의 Attribute를 포함한 새로운 Relation을 만들고 하나의 X side의 attribute를 Primary key로 설정**한다.

- X side가 Primary key가 되었기 때문에 **3NF를 만족**한다.

모든 2NF Relation은 Normalization process를 통해 3NF로 Decompose될 수 있다.

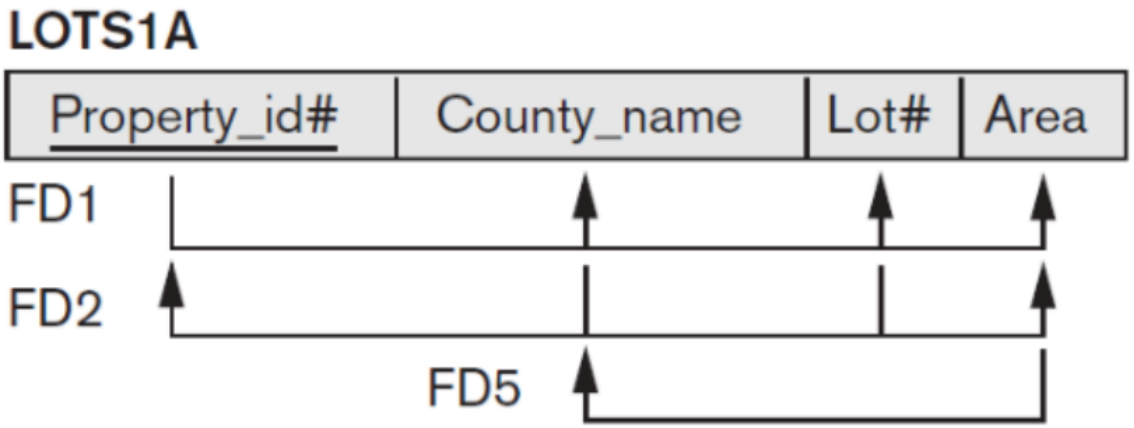
Boyce-Codd Normal Form (BCNF)

> **이전 단계의 NF를 모두 만족할 때, Relation R의 모든 FD $X \rightarrow A$에 대해 X가 Superkey인 경우**

- 3NF와 비교하였을 때, A가 Prime attribute여도 된다는 조건이 제거되었다.
- **BCNF가 3NF에 비해 더 까다롭다.**

Example

아래 상황에서 **FD5는 County_name이 Prime attribute이기 때문에 3NF는 만족하지만, BCNF는 만족하지 않는다.**



이를 해결하기 위해 **LOTS1A**를 두 개의 Relation으로 Decompose한다.

- **LOTS1AX:** FD5를 제거한다.
- **LOTS1AY:** FD5에 포함된 Attribute를 포함하는 Relation을 생성하고 X side를 PK로 설정한다.

모든 3NF Relation은 Normalization process를 통해 BCNF로 Decompose될 수 있다.

Summary

> **Normal form은 하위 level의 Normal form보다 Tighter condition을 가진다.**

- **Every 2NF relation are also in 1NF**

- **Every 3NF relation are also in 2NF**
- **Every BCNF relation are also in 3NF**

Good relational design

> **Database의 모든 Relation이 3NF 또는 BCNF가 되도록 하는 것**

- **Lossless join property**와 **Dependency preserve property**도 만족해야 한다.